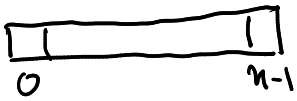
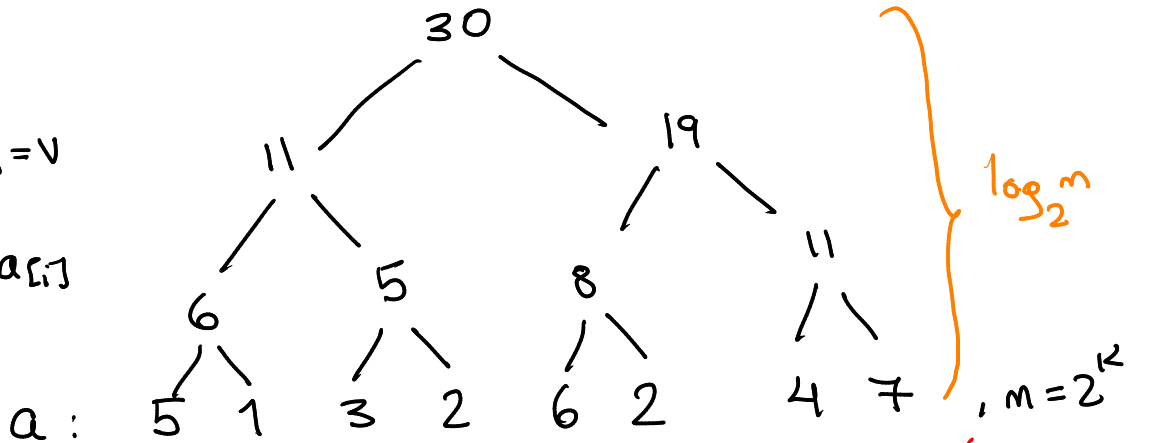
Segment Trees

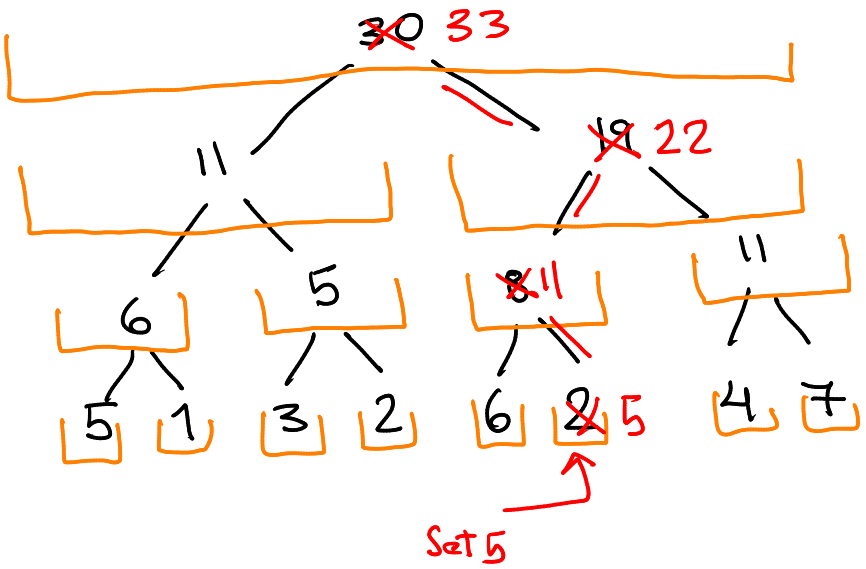
a: 

Set(i,v) // a[i]=v

Sum(l,r) // $\sum_{i=l}^{r-1} a[i]$

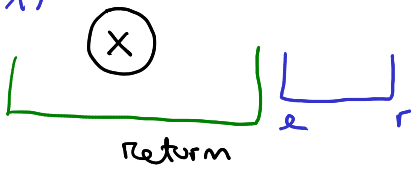


Set(i,v) O(lgm)

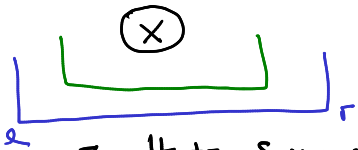


Sum(l,r) O(lgm)

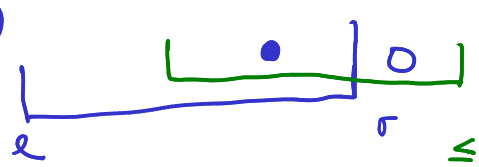
1)

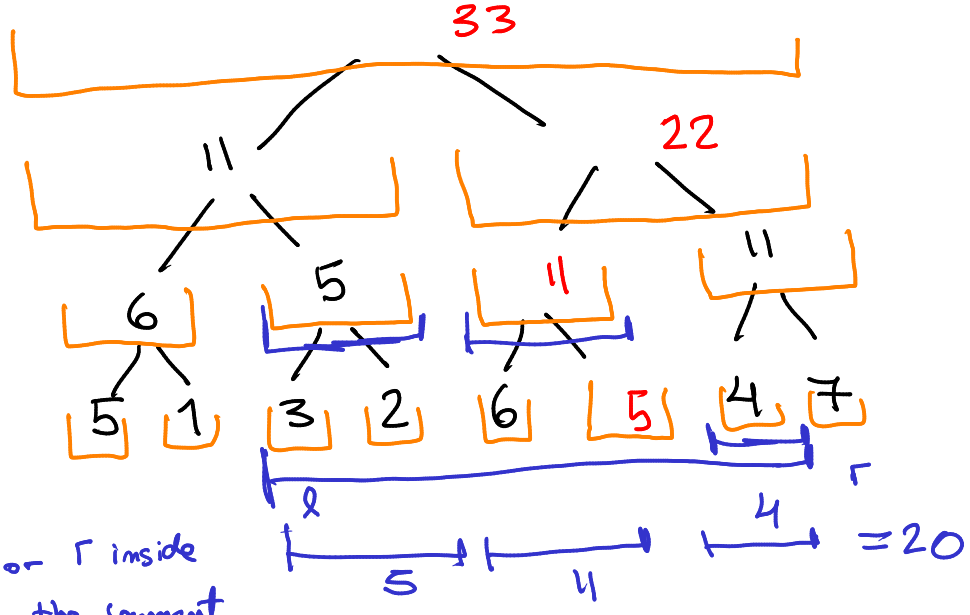


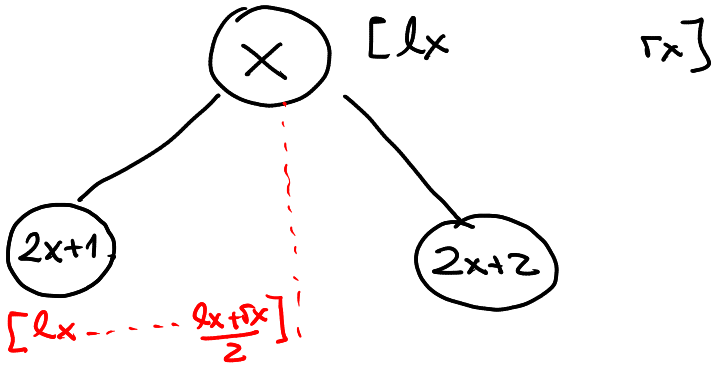
2)



3)





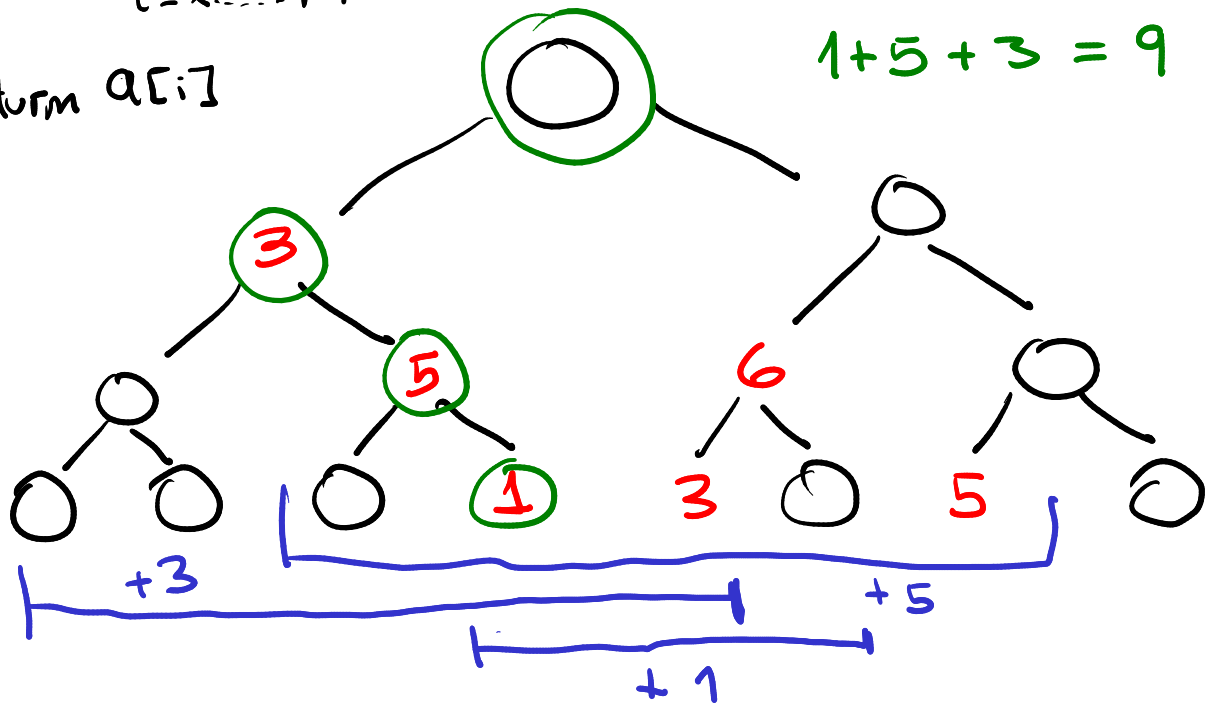



```
set(i, v, x, lx, rx):
  if rx - lx = 1:
      tree[x] = v
      return
  m = (lx + rx)/2
  if i < m:
      set(i, v, 2x+1, lx, m)
  else:
      set(i, v, 2x+2, m, rx)
  tree[x] = tree[2x+1] + tree[2x+2]
```

```
sum(l, r, x, lx, rx):
  if l ≥ rx or r ≤ lx:
      return 0
  if lx ≥ l and rx ≤ r:
      return tree[x]
  m = (lx + rx)/2
  s1 = sum(l, r, 2x+1, lx, m)
  s2 = sum(l, r, 2x+2, m, rx)
  return s1 + s2
```

Add on range

```
add(l, r, v) // a[i] += v
                i = l ... r-1
get(i) // return a[i]
```



```
add(l, r, v, x, lx, rx):
    if lx >= r || l >= rx:
        return
    if lx >= l && rx <= r:
        tree[x] += v
        return
    m = (lx + rx) / 2
    add(l, r, v, 2x+1, lx, m)
    add(l, r, v, 2x+2, m, rx)
```

```
get(i, x, lx, rx):
    if rx - lx = 1:
        return tree[x]
    m = (lx + rx) / 2
    if i < m:
        s = get(i, 2x+1, lx, m)
    else:
        s = get(i, 2x+2, m, rx)
    return s + tree[x]
```

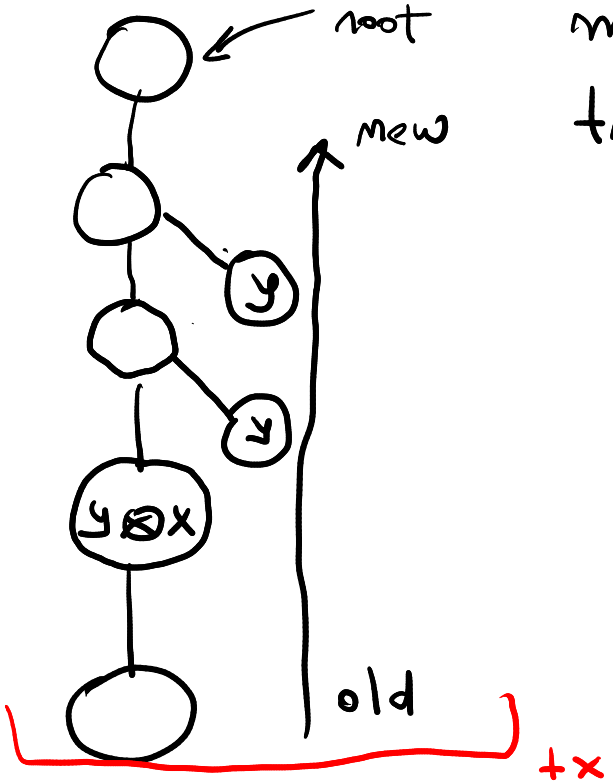
modify(l, r, v) // a[i] = a[i] ⊗ v

1) $(a[i] \otimes x) \otimes y = a[i] \otimes (x \otimes y)$ Associativity

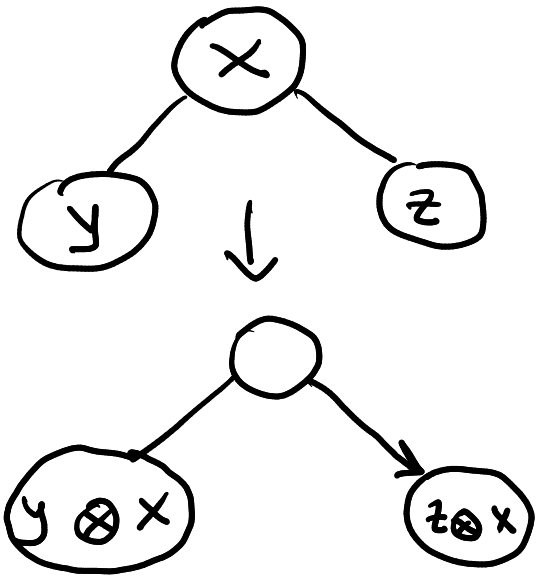
2) $(a[i] \otimes x) \otimes y = (a[i] \otimes y) \otimes x$ Commutativity

- modify similar to add iff 1) and 2)

Si no cumple 2): (Lazy propagation)



move the operations down to the tree.



Propagate (x, lx, rx)

Put in the first line in modify(add) and get.

```
if rx - lx = 1:
    return
tree[2x+1] = tree[2x+1] ⊗ tree[x]
tree[2x+2] = tree[2x+2] ⊗ tree[x]
tree[x] = 0
```

← NEUT

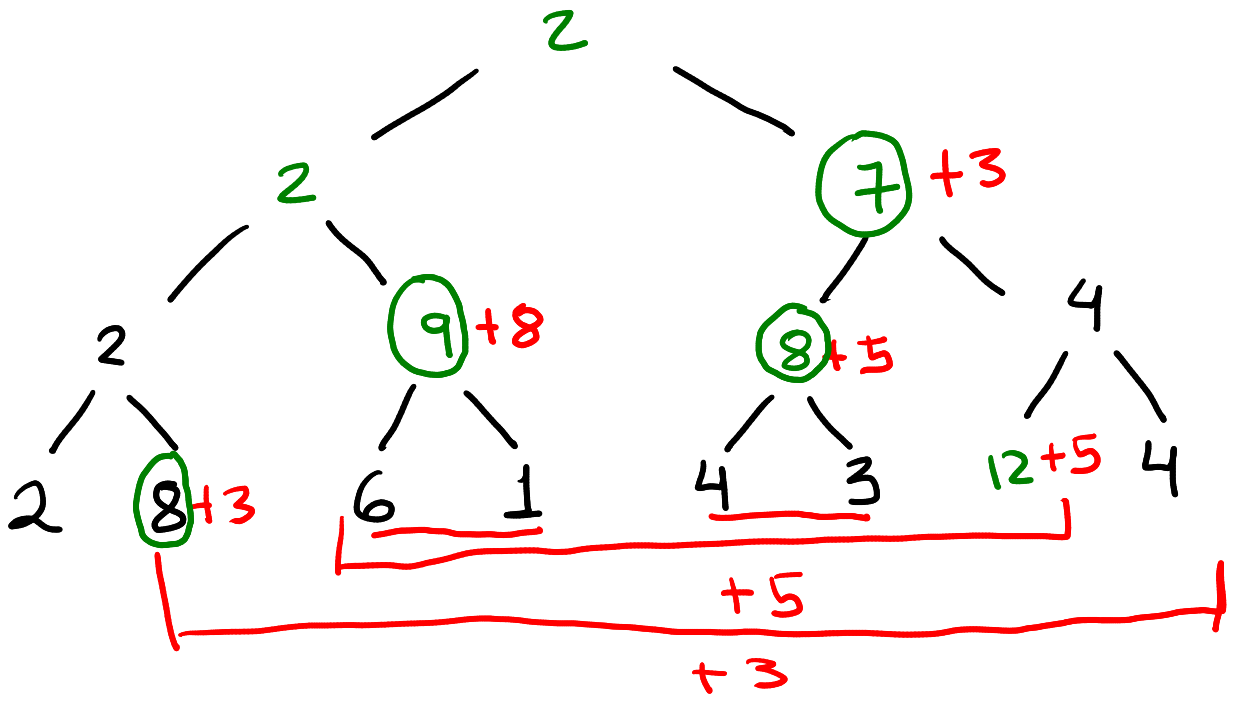
Modify Segment and CALC SEGMENT:

Ex: a

add(l, r, v) // a[i] ± v (modify) a[i] = a[i] ⊗ v

i = l, ..., r-1

min(l, r) // return min(a(l, ..., r-1)) (calc) a[l] ⊙ a[l+1] ⊙ ... ⊙ a[r-1]



```
add (l, r, v, x, lx, rx):
    propagate (x)
    if lx >= r || l >= rx :
        return
    if lx >= l and rx <= r:
        op[x] = op[x] ⊗ v
        val[x] = val[x] ⊗ v
        return
    m = (lx + rx)/2
    add (l, r, v, 2x+1, lx, m)
    add (l, r, v, 2x+2, m, rx)
    val[x] = (val[2x+1] ⊙ val[2x+2]) ⊗ op(x)
```

not necessary with propagate

```
min (l, r, x, lx, rx ):
    propagate(x, lx, rx)
    if lx >= r or l >= rx :
        return ∞ ← NEUT
    if lx >= l and rx <= r:
        return value[x]
    m = (lx + rx)/2
    m1 = min(l, r, 2x+1, lx, m)
    m2 = min (l, r, 2x+2, m, rx)
    return min(m1, m2) + op[x]
```

Modify (l, r, v) // $a[i] \otimes = v$

Calc (l, r) // $a[l] \odot a[l+1] \odot \ldots \odot a[r-1]$

$\otimes$: associative

$\odot$: associative

En el ejemplo $\min(x+v, y+v) = \min(x, y) + v$

$$(x \otimes v) \odot (y \otimes v) = (x \odot y) \otimes v$$

Distributivity.